

Algoritma dan Pemrograman Lanjut

Pertemuan 1: Pengantar Pemrograman Lanjut

Tipe Data Sekuensial

Hadian Mandala Putra, S.Si., M.Kom.

Universitas Hamzanwadi
Program Studi Teknik Komputer



Pendahuluan

- Tipe data sekuensial adalah struktur data yang menyimpan koleksi nilai.
- Berfungsi untuk menyimpan data dalam jumlah banyak dengan berbagai cara.
- Penting karena memudahkan pengelolaan data secara efisien dalam pemrograman.

List

Apa Itu List?

- List adalah salah satu tipe data sekuensial paling populer di Python.
- List bersifat **mutable**, artinya dapat diubah setelah dideklarasikan.
- List menyimpan kumpulan elemen yang bisa bertipe data sama atau berbeda.
- Karena fleksibilitasnya, list sangat sering digunakan dalam pemrograman Python.

Karakteristik List

- List bersifat **terurut** (ordered).
- Elemen diakses menggunakan indeks.
- List dapat menyimpan elemen dengan tipe data campuran.
- List bersifat **dinamis**, ukurannya bisa berubah.
- Mendukung operasi penambahan, penghapusan, dan pengubahan elemen.

Membuat List di Python

Contoh Kode List

```
1 # List kosong
2 data = []
3
4 # List berisi elemen
5 angka = [1, 2, 3, 4, 5]
6 campuran = [1, 'Python', True, 3.14]
```

Akses Elemen List

Contoh Akses Elemen

```
1 angka = [10, 20, 30, 40, 50]
2
3 # Akses berdasarkan indeks
4 print(angka[0]) # 10
5 print(angka[-1]) # 50
6
7 # Menggunakan slicing
8 print(angka[1:4]) # [20, 30, 40]
```

Menambah Elemen List

Contoh Menambah Elemen

```
1 data = [1, 2, 3]
2
3 # Menambah satu elemen
4 data.append(4)
5
6 # Menambah beberapa elemen
7 data.extend([5, 6, 7])
8
9 # Menambah di posisi tertentu
10 data.insert(1, 'Python')
11
12 print(data)
```

Menghapus Elemen List

Contoh Menghapus Elemen

```
1 data = [1, 2, 3, 4, 5]
2
3 # Menghapus berdasarkan nilai
4 data.remove(3)
5
6 # Menghapus berdasarkan indeks
7 del data[1]
8
9 # Menghapus dan mengambil elemen
10 elemen = data.pop()
11
12 print(data, elemen)
```

Mengubah Elemen List

Contoh Mengubah Elemen

```
1 data = ['a', 'b', 'c', 'd']
2
3 # Mengubah elemen pertama
4 data[0] = 'A'
5
6 # Mengubah sebagian menggunakan slicing
7 data[1:3] = ['X', 'Y']
8
9 print(data)
```

Operasi dan Metode List Lainnya

Contoh Operasi Lain

```
1 data = [3, 1, 4, 1, 5, 9]
2
3 # Menghitung elemen tertentu
4 print(data.count(1)) # 2
5
6 # Mencari indeks elemen
7 print(data.index(4)) # 2
8
9 # Membalik urutan
10 data.reverse()
11
12 # Mengurutkan list
13 data.sort()
14
15 print(data)
```

List Bersarang (Nested List)

Contoh Nested List

```
1 nested = [  
2     [1, 2, 3],  
3     ['a', 'b', 'c'],  
4     [True, False]  
5 ]  
6  
7 # Akses elemen di dalam list bersarang  
8 print(nested[1][2]) # 'c'
```

Konversi Tipe Data ke List

Contoh Konversi Tipe Data

```
1 # Dari string ke list
2 text = "Python"
3 lst = list(text)
4
5 # Dari tuple ke list
6 tpl = (1, 2, 3)
7 lst2 = list(tpl)
8
9 print(lst, lst2)
```

Keunggulan List

- Fleksibel: bisa menyimpan campuran tipe data.
- Dinamis: ukurannya bisa bertambah/berkurang.
- Kaya metode manipulasi data.

Kekurangan List

- Akses data lebih lambat dibanding tuple.
- Konsumsi memori lebih besar.

Kesimpulan

- List cocok untuk data yang sering berubah.
- Mendukung manipulasi data yang fleksibel.
- Menguasai list penting untuk Python Programmer.
- Untuk data tetap, pertimbangkan menggunakan tuple.

Tuple

Apa Itu Tuple?

- Tuple adalah salah satu tipe data sekuensial di Python.
- Berbeda dengan list, tuple bersifat **immutable** (tidak dapat diubah).
- Cocok digunakan untuk data tetap yang tidak perlu dimodifikasi.
- Memiliki performa lebih cepat dibandingkan list.

Karakteristik Tuple

- Bersifat **terurut** (ordered).
- Dapat menyimpan berbagai tipe data.
- Akses elemen menggunakan indeks.
- Mendukung **nested structure** (tuple di dalam tuple).
- Dapat digunakan sebagai key pada dictionary.

Cara Membuat Tuple

Contoh Kode Tuple

```
1 # Tuple kosong
2 kosong = ()
3
4 # Tuple berisi data
5 data = (1, 2, 3, 'Python', True)
6
7 # Tuple packing (tanpa kurung)
8 data2 = 1, 2, 3
9
10 # Tuple satu elemen (harus pakai koma)
11 single = (5,)
```

Mengakses Elemen Tuple

Contoh Kode Akses Elemen

```
1 data = ('a', 'b', 'c', 'd')  
2 print(data[0]) # 'a'  
3 print(data[-1]) # 'd'
```

Slicing dan Menghitung Elemen

Contoh Slicing dan Len

```
1 data = (10, 20, 30, 40, 50)
2 print(data[1:4]) # (20, 30, 40)
3 print(len(data)) # 5
```

Mengapa Immutable Penting?

- Mengurangi risiko perubahan data secara tidak sengaja.
- Meningkatkan keamanan data.
- Memungkinkan optimasi memori oleh Python.
- Tuple bisa menjadi key pada dictionary.

Fungsi dan Metode Tuple

Contoh Metode Tuple

```
1 data = (1, 2, 3, 2, 4)
2
3 # Menghitung jumlah elemen tertentu
4 print(data.count(2)) # 2
5
6 # Mencari indeks elemen pertama
7 print(data.index(4)) # 4
```

Kelebihan Tuple

- Lebih cepat diproses.
- Lebih hemat memori.
- Aman untuk data tetap.
- Bisa menjadi key di dictionary.

Kekurangan Tuple

- Tidak dapat dimodifikasi setelah dideklarasikan.
- Tidak memiliki metode manipulasi data seperti list.

Tuple Bersarang

Contoh Nested Tuple

```
1 nested = (1, (2, 3), (4, (5, 6)))  
2 print(nested[1])    # (2, 3)  
3 print(nested[2][1]) # (5, 6)
```

Konversi Tuple dan List

Contoh Konversi

```
1 lst = [1, 2, 3]
2 tpl = tuple(lst)
3 lst2 = list(tpl)
```

Tuple sebagai Return Function

Contoh Multi-return

```
1 def hitung(p, l):  
2     return p*l, 2*(p+l)  
3  
4 luas, keliling = hitung(5, 3)
```

Tuple sebagai Key Dictionary

Contoh Tuple Key

```
1 koordinat = {}  
2 koordinat[(1, 2)] = 'Posisi A'  
3 koordinat[(3, 4)] = 'Posisi B'  
4 print(koordinat)
```

Studi Kasus: Data Tetap

- Menyimpan nama hari dalam seminggu.

Contoh Kasus

```
1 hari = ('Senin', 'Selasa', 'Rabu', 'Kamis', 'Jumat', 'Sabtu',  
         'Minggu')
```

Kesimpulan

- Tuple cocok untuk data tetap.
- Immutability meningkatkan keamanan.
- Performanya lebih baik dibanding list.
- Kunci dictionary yang baik.

Set

Apa itu Set?

Set adalah salah satu tipe data sekuensial di Python yang digunakan untuk menyimpan sekumpulan item yang bersifat unik dan tidak berurutan. Set tidak menerima elemen duplikat, menjadikannya tipe data yang ideal untuk operasi berbasis himpunan seperti pencarian item unik, perpotongan, atau gabungan.

Karakteristik utama set:

- Elemen di dalam set bersifat unik (tidak boleh duplikat).
- Elemen tidak memiliki indeks (tidak dapat diakses seperti list atau tuple).
- Set bersifat mutable (dapat diubah isinya).
- Set menggunakan kurung kurawal {}.

Membuat Set

Membuat set sangat mudah menggunakan kurung kurawal atau fungsi bawaan `set()`.

Contoh Kode Membuat Set

```
1 my_set = {1, 2, 3, 'Python', 'Data'}  
2 print(my_set)  
3  
4 # atau menggunakan fungsi set()  
5 another_set = set([3, 4, 5, 6])  
6 print(another_set)
```

Mengapa Menggunakan Set?

Set sangat bermanfaat ketika kita ingin:

- Menghapus data duplikat dari koleksi.
- Melakukan operasi himpunan (union, intersection, difference).
- Menyimpan koleksi item unik tanpa peduli urutan.

Set lebih efisien untuk pencarian karena diimplementasikan menggunakan struktur hash table.

Menambah Elemen Set

Kita bisa menambahkan item baru ke dalam set menggunakan metode `add()`.

Contoh Menambah Elemen Set

```
1 my_set = {1, 2, 3}
2 my_set.add(4)
3 print(my_set)
4
5 # Menambah elemen string
6 my_set.add('Python')
7 print(my_set)
```

Menghapus Elemen Set

Terdapat beberapa cara menghapus elemen dari set:

- `remove()` - Menghapus item (error jika tidak ditemukan)
- `discard()` - Menghapus item (tidak error jika tidak ditemukan)
- `clear()` - Menghapus semua item

Contoh Menghapus Elemen Set

```
1 my_set = {1, 2, 3, 'Python'}
2 my_set.remove(2)
3 print(my_set)
4
5 my_set.discard('Java') # Tidak error meski tidak ada
6 my_set.clear()
7 print(my_set) # Hasilnya set kosong
```

Operasi Himpunan dengan Set

Python menyediakan berbagai operasi himpunan pada set:

- Union (`|` atau `union()`)
- Intersection (`&` atau `intersection()`)
- Difference (`-` atau `difference()`)
- Symmetric Difference (`^` atau `symmetric_difference()`)

Contoh Operasi Himpunan

Contoh Operasi Himpunan

```
1  A = {1, 2, 3, 4}
2  B = {3, 4, 5, 6}
3
4  # Union
5  print(A | B)
6
7  # Intersection
8  print(A & B)
9
10 # Difference
11 print(A - B)
12
13 # Symmetric Difference
14 print(A ^ B)
```

Pengecekan Keanggotaan

Kita bisa menggunakan operator `in` dan `not in` untuk mengecek apakah elemen ada dalam set.

Contoh Pengecekan Keanggotaan

```
1 my_set = {'Python', 'Java', 'C++'}  
2 if 'Python' in my_set:  
3     print('Python ada di dalam set')
```

Studi Kasus

Kasus: Anda memiliki data daftar nama mahasiswa, beberapa nama terduplikat. Solusi: Gunakan set untuk menyimpan nama unik.

Contoh Studi Kasus

```
1 mahasiswa = ['Ali', 'Budi', 'Citra', 'Ali', 'Dewi']  
2 set_mahasiswa = set(mahasiswa)  
3 print(set_mahasiswa)
```

Kelebihan dan Kekurangan Set

Kelebihan:

- Penyimpanan item unik secara otomatis.
- Performa pencarian sangat cepat.
- Mendukung operasi himpunan.

Kekurangan:

- Tidak bisa menyimpan item yang mutable (contoh: list atau dictionary).
- Tidak berurutan, sulit diakses berdasarkan indeks.

Kesimpulan

Gunakan set jika:

- Anda butuh kumpulan item unik.
- Anda tidak peduli urutan.
- Anda perlu operasi himpunan.

Hindari set jika urutan atau indeks penting.

Dictionary

Pengertian Dictionary

Dictionary adalah struktur data di Python yang menyimpan data dalam bentuk pasangan **key-value**.

Setiap elemen dictionary terdiri dari:

- **Key:** Identifier unik yang tidak boleh duplikat.
- **Value:** Nilai yang berasosiasi dengan key, dapat berupa tipe data apapun.

Dictionary sangat efisien untuk pencarian berdasarkan key, mirip seperti kamus dunia nyata yang memetakan kata ke definisi.

Deklarasi Dictionary

Membuat dictionary bisa menggunakan kurung kurawal {} atau fungsi dict().

Contoh Kode Membuat Dictionary

```
1  # Dictionary kosong
2  data = {}
3
4  # Dictionary dengan pasangan key-value
5  mahasiswa = {
6      "nim": "220001",
7      "nama": "Budi",
8      "jurusan": "Informatika"
9  }
10 print(mahasiswa)
```

Karakteristik Dictionary

- Key harus unik.
- Value bisa bertipe apapun (string, integer, list, bahkan dictionary lain).
- Dictionary bersifat mutable (dapat diubah isinya).
- Elemen tidak memiliki urutan tetap.

Mengakses Elemen Dictionary

Elemen diakses menggunakan key.

Contoh Mengakses Elemen

```
1 mahasiswa = {"nim": "220001", "nama": "Budi"}  
2 print(mahasiswa["nama"]) # Output: Budi
```

Jika key tidak ditemukan, Python akan melempar error `KeyError`. Untuk menghindari, gunakan metode `get()`:

```
print(mahasiswa.get("alamat", "Alamat belum diisi"))
```

Menambah dan Mengubah Elemen

Menambah cukup dengan memberi key baru, mengubah cukup dengan mengisi ulang value.

Contoh Menambah dan Mengubah Elemen

```
1 # Menambah
2 mahasiswa["alamat"] = "Jl. Merdeka"
3
4 # Mengubah
5 mahasiswa["nama"] = "Budi Santoso"
6
7 print(mahasiswa)
```

Menghapus Elemen

Gunakan metode:

- `pop(key)` - Menghapus berdasarkan key.
- `popitem()` - Menghapus pasangan terakhir.
- `clear()` - Menghapus seluruh item.

Contoh Menghapus Elemen

```
1 mahasiswa.pop("alamat")
2 print(mahasiswa)
3
4 mahasiswa.clear() # Kosongkan dictionary
5 print(mahasiswa)
```

Iterasi Dictionary

Looping bisa langsung melalui:

- `keys()` - Melintasi semua key.
- `values()` - Melintasi semua value.
- `items()` - Melintasi pasangan key-value.

Contoh Iterasi

Contoh Iterasi

```
1 for key, value in mahasiswa.items():  
2     print(f"{key}: {value}")
```

Kelebihan Dictionary

- Akses data cepat ($O(1)$) menggunakan key.
- Menyimpan data kompleks dengan struktur key-value.
- Mendukung nested structure (dictionary bersarang).

Kekurangan Dictionary

- Menggunakan lebih banyak memori dibanding list atau tuple.
- Tidak memiliki urutan tetap.

Studi Kasus

Kasus: Menyimpan data mahasiswa yang terdiri dari NIM, nama, dan IPK.

Contoh Studi Kasus

```
1 mahasiswa = {  
2     "nim": "220001",  
3     "nama": "Budi",  
4     "ipk": 3.75  
5 }  
6 print(f"{mahasiswa['nama']} memiliki IPK {mahasiswa['ipk']}")
```

Contoh Dictionary Bersarang

Nested Dictionary memungkinkan penyimpanan struktur data lebih kompleks.

Contoh Nested Dictionary

```
1 kampus = {  
2     "fakultas": {  
3         "teknik": ["Informatika", "Elektro"],  
4         "ekonomi": ["Akuntansi", "Manajemen"]  
5     }  
6 }  
7 print(kampus["fakultas"]["teknik"])
```

Kesimpulan

Gunakan dictionary saat:

- Data berbentuk pasangan key-value.
- Data perlu diakses langsung via key.
- Anda ingin menyusun struktur data kompleks.

Dictionary adalah solusi optimal untuk merepresentasikan data berlabel.

Perbandingan Singkat Tipe Data Sekuensial

Tipe Data	Urutan	Mutable	Duplikasi	Sintaks
List	Ya	Ya	Ya	[]
Tuple	Ya	Tidak	Ya	()
Set	Tidak	Ya	Tidak	{ }
Dictionary	Tidak (Key)	Ya (Value)	Key Unik	{ }

Kerjakan seluruh soal berikut dalam bentuk kode Python. Setiap soal merupakan studi kasus yang harus diselesaikan menggunakan konsep yang sesuai: **List, Tuple, Set, atau Dictionary**. Tulis jawaban Anda di file Python terpisah atau langsung tuliskan di kertas jawaban.

Studi Kasus 1: Pengolahan Nilai Mahasiswa

Buatlah program yang membaca daftar nilai mahasiswa menggunakan List. Hitung rata-rata nilai, nilai tertinggi, dan nilai terendah.

Studi Kasus 2: Manipulasi Daftar Produk

Diberikan daftar nama produk. Buatlah program untuk menambah produk baru, menghapus produk tertentu, dan mencetak daftar produk yang sudah diurutkan.

Studi Kasus 3: Data Koordinat

Simpan data koordinat (x, y) dari 5 titik menggunakan Tuple. Tampilkan semua titik, lalu tampilkan titik dengan jarak terjauh dari titik asal $(0, 0)$.

Studi Kasus 4: Data Buku

Data buku disimpan sebagai tuple berisi (judul, penulis, tahun_terbit). Buatlah list berisi 5 tuple buku, lalu cetak semua buku yang terbit setelah tahun 2015.

Studi Kasus 5: Data Unik Mahasiswa

Simpan NIM mahasiswa dalam Set agar tidak ada duplikasi. Buatlah program yang dapat menambah NIM baru dan menampilkan seluruh NIM.

Studi Kasus 6: Operasi Himpunan

Diberikan dua himpunan mata kuliah: wajib dan pilihan. Buatlah program yang mencetak irisan, gabungan, dan selisih kedua himpunan tersebut.

Studi Kasus 7: Data Mahasiswa

Simpan data mahasiswa dalam Dictionary dengan format: {nim: nama}. Buat fungsi untuk menambah, menghapus, dan menampilkan semua data.

Studi Kasus 8: Transaksi Penjualan

Buat dictionary yang menyimpan nama produk sebagai key dan jumlah stok sebagai value. Program dapat menambah produk baru, mengurangi stok saat produk terjual, dan mencetak semua stok.

Studi Kasus 9: Data Nilai Mahasiswa

Simpan data nilai mahasiswa dalam List of Dictionary. Setiap dictionary menyimpan: nim, nama, dan nilai. Cetak semua data dan hitung rata-rata nilai kelas.

Studi Kasus 10: Jadwal Mata Kuliah

Simpan jadwal kuliah dalam Dictionary, dengan key adalah nama mata kuliah, dan value adalah tuple berisi (hari, jam). Cetak seluruh jadwal.

Studi Kasus 11: Matriks Nilai

Buatlah matriks 3x3 berisi nilai-nilai integer. Tampilkan matriks dalam bentuk tabel dan hitung total serta rata-rata setiap baris.

Studi Kasus 12: Mahasiswa yang Mengikuti UKM

Terdapat dua himpunan mahasiswa yang mengikuti UKM Seni dan UKM Olahraga. Cetak mahasiswa yang mengikuti kedua UKM, hanya seni, hanya olahraga, dan semua mahasiswa.

Studi Kasus 13: Data KRS Mahasiswa

Simpan data KRS mahasiswa dalam Dictionary, dengan format: {nim: {nama: '...', mata_kuliah: [list matkul]}}. Cetak KRS semua mahasiswa.

Studi Kasus 14: Data Pemesanan

Simpan data pemesanan hotel berupa List of Tuple. Setiap tuple berisi (nama, jenis_kamar, lama_inap). Tampilkan semua data dan hitung total tamu.

Studi Kasus 15: Data Inventaris Barang

Simpan data barang dalam Dictionary dengan format: {kode_barang: {nama: '...', jumlah: x}}. Buat fungsi untuk menambah, menghapus, memperbarui jumlah barang, dan mencetak semua data.

- ➊ Seluruh kode harus valid Python 3 dan dapat dijalankan tanpa error. Tulis komentar pada setiap baris penting agar mudah dipahami.
- ➋ Tugas dikumpulkan 1 minggu setelah tugas diberikan
- ➌ Tuliskan NIM, Nama dan Kelas pada setiap lembar jawaban
- ➍ Jawaban dikumpulkan dalam bentuk file pdf hasil scan tulisan tangan sendiri dengan format penamaan file: NIM_NAMA_T1